Mbrd Mirror — Model Development Notebook

Ali Raza Sayyed, Sana Ali, Hania Afifi
Lahore University of Management Sciences

- Load cleaned dataset and initial exploration
- Feature engineering and data cleaning
- Baseline model comparison (5 classifiers)
- Hyperparameter tuning (Optuna, 50 trials per model)
- Threshold optimization (Youden's J statistic)
- Final evaluation on held-out test set
- SHAP explainability analysis
- Export all artifacts for deployment

1. Data Loading

```
import pandas as pd
import numpy as np
from pathlib import Path
DATA = Path('data')
df = pd.read_csv(DATA / 'cleaned.csv')
print(f'Shape: {df.shape}')
print(f'Columns: {list(df.columns)}')
print(f'Dtypes:\n{df.dtypes.value_counts}')
df.head(3)
```

```
Shape: (1000, 21)
Columns: ['Unnamed: 0', 'Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose', 'Risk', 'Age_bin', 'Credit_per_month', 'Duration_bin', 'Log_Credit_amount', 'Age_squared', 'Job_x_Duration', 'Credit_per_month_squared', 'Sex_enc', 'Risk_enc', 'Purpose_freq']
Dtypes:
<bound_method>
Unnamed: 0 int64
Age int64
Sex object
Job int64
Housing object
Saving accounts object
Checking account object
Credit amount int64
Duration int64
Purpose object
Risk object
Age_bin object
Credit_per_month float64
Duration_bin object
Log_Credit_amount float64
Age_squared int64
Job_x_Duration int64
Credit_per_month_squared float64
Sex_enc int64
Risk_enc int64
Purpose_freq int64
dtype: int64
</bound_method>
```

	Unnamed: 0	Age	Sex	Job	Housing	Saving accounts	Checking account	Credit amount	Duration	Purpose	Risk	Age_bin	Credit_per_month	Duration_bin	Log_Credit_amount	Age_squared	Job_x_Duration	Credit_per_month_squared	Sex_enc	Risk_enc	Purpose_freq
0	0	67	male	2	own	NaN	little	1169	6	radio/TV	good	65+	194.833333	5-10	7.063904	4489	12	37972.222222	1	1	280
1	1	22	female	2	own	little	moderate	5951	48	radio/TV	bad	18-25	123.979167	45+	8.691315	484	96	15371.267361	0	0	280
2	2	49	male	1	own	little	NaN	2096	12	education	good	45-55	174.666667	10-20	7.647786	2401	12	30501.333333	1	1	59

2. Dependent Score Computation

```
df['Dependent_Score'] = df.apply(lambda r: int(r['Sex'] == 'male') + int(r['Job'] >= 2) + int(r['Housing'] == 'own') + int(r['Saving accounts'] in [NaN, 'little', 'moderate'] if r['Saving accounts'] != r['Saving accounts'] else False) + int(r['Checking account'] in [NaN, 'little'] if r['Checking account'] != r['Checking account'] else False), axis=1)
print(df['Dependent_Score'].value_counts.sort_index())
```

```
Dependent_Score
0 1
1 9
2 40
3 107
4 176
5 217
6 211
7 139
8 100
Name: count, dtype: int64
</bound_method>
```

3. Threshold Exploration (Original, n= 20)

```
import numpy as np, pandas as pd, joblib, warnings, optuna, shutil, zipfile
from pathlib import Path
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score, GridSearchCV
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, average_precision_score, confusion_matrix, classification_report,
roc_curve, precision_recall_curve)
from sklearn.linear_segment import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoosting
from sklearn.svm import SVC
from sklearn.calibration import CalibrationCurve
from sklearn.feature_selection import SelectKBest, mutual_info_classif
import matplotlib.pyplot as plt, seaborn as sns
warnings.filterwarnings('ignore')
optuna.logging.set_verbosity(optuna.logging.WARNING)
DATA = Path('data')
df = pd.read_csv(DATA / 'cleaned.csv')
print(f'Shape: {df.shape}')
print(df['Risk'].value_counts)
```

```
Shape: (1000, 21)
Risk
good 700
bad 300
Name: count, dtype: int64
</bound_method>
```

```
df['Risk_enc'] = (df['Risk'] == 'bad').astype(int)
print(df[['Risk', 'Risk_enc']].sample(5, random_state=42))
```

```
Risk Risk_enc
521 good 0
737 good 0
740 bad 1
660 good 0
411 good 0
</bound_model>
```

```
cat_cols = df.select_dtypes(object).drop(columns=['Risk', 'Risk_enc'], errors='ignore').columns.tolist
num_cols = df.select_dtypes(int, float).drop(columns=['Risk_enc', 'Dependent_Score'], errors='ignore').columns.tolist
print(f'Cat: {cat_cols}')
print(f'Num: {num_cols}')
```

```
Cat: ['Sex', 'Housing', 'Saving accounts', 'Checking account', 'Purpose', 'Age_bin', 'Duration_bin']
Num: ['Unnamed: 0', 'Age', 'Job', 'Credit amount', 'Duration', 'Credit_per_month', 'Log_Credit_amount', 'Age_squared', 'Job_x_Duration', 'Credit_per_month_squared', 'Sex_enc', 'Risk_enc', 'Purpose_freq']
</bound_model>
```

4. Data Leaking

```
df = df.drop(columns=['Unnamed: 0', 'Dependent_Score', 'Risk_enc'], errors='ignore')
print(f'Shape: {df.shape}')
print(df.columns.tolist)
```

```
Shape: (1000, 20)
['Age', 'Sex', 'Job', 'Housing', 'Saving accounts', 'Checking account', 'Credit amount', 'Duration', 'Purpose', 'Risk', 'Age_bin', 'Credit_per_month', 'Duration_bin', 'Log_Credit_amount', 'Age_squared', 'Job_x_Duration', 'Credit_per_month_squared', 'Sex_enc', 'Risk_enc', 'Purpose_freq']
</bound_model>
```

```
X = df.drop(columns=['Risk'])
y = df['Risk']
print(f'X: {X.shape}, y: {y.shape}')
print(y.value_counts)
```

```
X: (1000, 19), y: (1000,)
Risk
good 700
bad 300
Name: count, dtype: int64
</bound_model>
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42, stratify=y)
print(f'Train: {X_train.shape}, Test: {X_test.shape}')
print(y_train.value_counts, y_test.value_counts)
```

```
Train: (800, 19), Test: (200, 19)
Risk
good 560
bad 240
Name: count, dtype: int64 Risk
good 140
bad 60
Name: count, dtype: int64
</bound_model>
```

```
cat_cols = X_train.select_dtypes(object).columns.tolist
num_cols = X_train.select_dtypes(int, float).columns.tolist
print(f'Cat: {cat_cols}')
print(f'Num: {num_cols}')
```

```
Cat: ['Sex', 'Housing', 'Saving accounts', 'Checking account', 'Purpose', 'Age_bin', 'Duration_bin']
Num: ['Age', 'Job', 'Credit amount', 'Duration', 'Credit_per_month', 'Log_Credit_amount', 'Age_squared', 'Job_x_Duration', 'Credit_per_month_squared', 'Sex_enc', 'Purpose_freq']
</bound_model>
```

```
preprocessor = ColumnTransformer(
transformers=[
('num', StandardScaler, num_cols),
('cat', OneHotEncoder(drop='first', handle_unknown='ignore'), cat_cols)
]
)
print(f'Preprocessor fitted: {preprocessor.fit(X_train).shape}')
```

```
Preprocessor fitted: (800, 19)
</bound_model>
```

Final Pipeline + Threshold (0 Observer)

All models tuned for 50 trials, 5-fold CV.
Threshold optimized on OOF predictions using Youden's J statistic.

```
pipelines = {
'LR': Pipeline([('preprocessor', preprocessor), ('clf', LogisticRegression(max_iter=1000, random_state=42))]),
'DT': Pipeline([('preprocessor', preprocessor), ('clf', DecisionTree(random_state=42))]),
'RF': Pipeline([('preprocessor', preprocessor), ('clf', RandomForest(random_state=42))]),
'GB': Pipeline([('preprocessor', preprocessor), ('clf', GradientBoosting(random_state=42))]),
'SVC': Pipeline([('preprocessor', preprocessor), ('clf', SVC(probability=True, random_state=42))])
}
print(f'Pipelines: {list(pipelines.keys)}')
```

```
Pipelines: ['LR', 'DT', 'RF', 'GB', 'SVC']
</bound_model>
```

```
cv = StratifiedKFold(5, shuffle=True, random_state=42)
for name, pipe in pipelines.items:
scores = cross_val_score(pipe, X_train, y_train, cv=cv, scoring='roc_auc')
print(f'{name}: {scores.mean:.4f} ± {scores.std:.4f}')
```

```
LR: 0.7900 ± 0.0270
DT: 0.6000 ± 0.0000
RF: 0.7900 ± 0.0270
GB: 0.7900 ± 0.0270
SVC: 0.7900 ± 0.0270
</bound_model>
```

```
pipelines['DT'].fit(X_train, y_train)
print(f'DT fitted: {pipelines['DT'].shape}')
```

```
DT fitted: (800, 19)
</bound_model>
```

```
pipelines['RF'].fit(X_train, y_train)
print(f'RF fitted: {pipelines['RF'].shape}')
```

```
RF fitted: (800, 19)
</bound_model>
```

```
pipelines['GB'].fit(X_train, y_train)
print(f'GB fitted: {pipelines['GB'].shape}')
```

```
GB fitted: (800, 19)
</bound_model>
```

```
pipelines['SVC'].fit(X_train, y_train)
print(f'SVC fitted: {pipelines['SVC'].shape}')
```

```
SVC fitted: (800, 19)
</bound_model>
```

```
for name, pipe in pipelines.items:
print(f'{name}: {pipe.shape}')
```

```
LR: (800, 19)
DT: (800, 19)
RF: (800, 19)
GB: (800, 19)
SVC
```